VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgavi-590018

A Project Report On

"FIRE DETECTOR IOT SYSTEM WITH EXTINGUISHER CLASS RECOMMENDATION"

Submitted in the partial fulfillment for the award of the Bachelor of Engineering degree in Computer Science and Engineering

Submitted By

DISHANTH J	**4AD22CS028**
HARSHITH K V	**4AD22CS034**
HARSHITH R	**4AD22CS035**
JITHIN SIMON	**4AD22CS039**

Under the guidance of

Mrs. Roopa B
Assistant Professor
Department of Computer Science & Engineering

Department of Computer Science & Engineering
ATME College of Engineering
13th Kilometer, Mysore – Kanakapura - Bangalore Road, Mysore-570028

2025-2026

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgavi-590018

Department of Computer Science and Engineering

ATME College of Engineering

CERTIFICATE

This is to certify that the Project Work entitled **"FIRE DETECTOR IOT SYSTEM WITH EXTINGUISHER CLASS RECOMMENDATION"** is the Bonafide work carried out by **DISHANTH J (4AD22CS028), HARSHITH K V (4AD22CS034), HARSHITH R (4AD22CS035)** and **JITHIN SIMON (4AD22CS039)** in partial fulfillment for the award of degree of Bachelor of Engineering in Computer Science and Engineering from Visvesvaraya Technological University, Belagavi during the year 2025-2026. The report has been approved and satisfies the academic requirement with respect to Project Work prescribed for Bachelor of Engineering degree.

Signature of Guide	Signature of HOD	Signature of Principal
Mrs. Roopa B	**Dr. Puttegowda D**	**Dr. L Basavaraj**
Assistant Professor	Professor & Head	Principal

External Viva

Name of Examiners	**Signature with Date**
1…….	**……..**
2…….	**……..**

ACKNOWLEDGEMENT

The successful completion of our work would be incomplete without the mention of the names of the people who have made it possible. We are indebted to several individuals who have helped us to complete our project.

We are thankful to **Dr. L Basavaraj**, **Principal**, ATME College of Engineering for having granted us permission and extended full use of the college facilities to carry out our project successfully.

We express our profound gratitude to **Dr. Puttegowda D,** Professor & Head, Department of Computer Science and Engineering for his consistent co-operation and support.

At the outset we express our profound gratitude to our guide **Mrs. Roopa B,** Assistant Professor, Department of Computer Science and Engineering for her consistent co-operation and support.

We are greatly indebted to our project coordinator **Dr. J V Gorabal, Professor,** and **Dr. Shilpa B L, Associate Professor**, Department of Computer Science and Engineering for Their timely inquiries into the progress of the project

We also thank all teaching and non-teaching staff members of the department for their valuable assistance throughout our academic tenure.

Lastly, we would like to thank our family and friends for their cooperation and support for successful completion of our project.

DISHANTH J (4AD22CS028)
HARSHITH K V (4AD22CS034)
HARSHITH R (4AD22CS035)
JITHIN SIMON (4AD22CS039)

ABSTRACT

Fire outbreaks present significant threats to human life, property, and the environment, making early detection and timely response crucial. This project proposes an Internet of Things (IoT)-based fire detection system that not only identifies fire incidents but also identifies objects and recommends the appropriate class of fire extinguisher for suppression. The system integrates camera, gas sensor and temperature sensors, with IoT connectivity to monitor the environment and detect abnormal conditions. A deep learning model based on YOLOv8 is employed to detect fire regions and recognize burning objects, enabling accurate fire classification and extinguisher selection. Once a fire is detected, the system transmits data including fire location and extinguisher type to a central server, which immediately notifies users and emergency responders via smartphone applications.

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page No.
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
	TABLE OF CONTENTS	iii-iv
	LIST OF FIGURES	v
Chapter 1	**Introduction**	**01-02**
1.1	Overview	01
1.2	Existing System	01
1.3	Problem Statement	01-02
1.4	Proposed System	02
1.5	Advantages Over Current System	02
Chapter 2	**Literature Survey**	**03-05**
2.1	Survey Papers	03-04
2.2	Limitations of Existing Fire Detection System	04-05
Chapter 3	**Methodology**	**06-15**
3.1	System Requirements	06-10
3.2	Feasibility Study	10-11
3.3	System Architecture	11-15
Chapter 4	**Implementation**	**16-28**
4.1	Data Preparation	16-17
4.2	Model Training	17-20
4.3	Fire Detection	20-21

TABLE OF CONTENTS

4.4	Classification and Recommendation	21-25
4.5	System Testing	25-28
Chapter 5	**Results**	**29-36**
5.1	Comparative Study	29-30
5.2	Model Evaluation	30-32
5.3	Snapshots	33-36
Chapter 6	**Conclusion**	37
	References	38

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	Hardware Interconnection Diagram	13
3.2	System Architecture	15
4.1	Top-1 and Top-5 Accuracy per Epoch	18
4.2	Evaluation Metrics Graphs	19
4.3	System Flow	24
5.1	Confusion Matrix	30
5.2	True Label Images	31
5.3	Predicted Images	32
5.4	Login Screen	33
5.5	Create Account Screen	33
5.6	System Safe Dashboard	34
5.7	Danger Detected Dashboard	34
5.8	Incident History Screen	35
5.9	My Profile Screen	35
5.10	Fire Analysis (Upload Screen)	36
5.11	Fire Analysis (Results Screen)	36
5.12	Emergency Alert Email Notification	36

Chapter 1

INTRODUCTION

1.1 Overview

Fire incidents continue to pose serious threats to human life, property, and the environment, especially in densely populated regions and sensitive ecological zones. Traditional fire alarm systems that rely solely on smoke or heat sensors often suffer from delayed detection, limited monitoring capability, and frequent false alarms. With advancements in the Internet of Things (IoT), intelligent sensors, and deep learning, modern fire detection systems have become more efficient, reliable, and context-aware. Recent studies have demonstrated the potential of integrating IoT devices with artificial intelligence to achieve smarter fire detection, object identification, early warning, and automated response support. Building on these developments, this work presents an IoT-based fire detection framework enhanced with deep learning for fire localization, object recognition, and fire extinguisher class recommendation.

1.2 Existing System

Existing fire detection systems predominantly rely on conventional smoke detectors and heat sensors that operate based on physical changes in the environment. These systems, although widely deployed, encounter significant limitations such as slow detection due to the time required for smoke or heat to reach the sensor, as well as high susceptibility to false alarms caused by dust, humidity, or sudden temperature variations. They also lack environmental awareness, meaning they are unable to identify the type of object burning or classify the nature of the fire. Even though recent IoT-based systems have introduced real-time alerting and remote monitoring, most of them still focus solely on detecting the presence of fire without providing deeper insights. Furthermore, many existing research works on vision-based fire detection concentrate only on image or video analysis without combining sensor data or offering appropriate suppression recommendations. As a result, current systems fall short of delivering an integrated and intelligent fire detection solution.

1.3 Problem Statement

Despite substantial progress in IoT-based monitoring and deep learning-driven fire detection, a major gap persists in integrating these technologies into a unified, intelligent system. Current solutions often address only one aspect of the fire detection pipeline: either they rely

exclusively on sensor-based detection, which may be slow or inaccurate, or they emphasize video-based identification, which may struggle under low-light or obstructed conditions. Moreover, most existing systems do not provide users with guidance on selecting the correct fire extinguisher for the detected scenario, which is crucial for effective and safe fire suppression. Therefore, there is a pressing need for a real-time, multi-sensor, AI-assisted fire detection system that combines environmental sensing, deep learning-based object identification, and automated extinguisher class recommendation to support rapid and informed emergency response.

1.4 Proposed System

The proposed system addresses these limitations by introducing an IoT-enabled intelligent fire detection framework that integrates temperature and gas sensors with a camera module and cloud connectivity. At the core of the system lies a YOLOv8 deep learning model trained to detect fire regions and recognize the type of object involved in the fire. The sensors continuously monitor environmental conditions, while the camera provides real-time visual data for analysis.

The integration of sensor readings with deep learning ensures comprehensive and accurate detection even in complex environments. Once the system identifies a fire, it classifies the burning object and automatically recommends the appropriate fire extinguisher class, such as Class A for solid combustibles or Class B for flammable liquids. The detected information, including fire severity, location, and recommended extinguisher type, is immediately transmitted to a central server or user interface, enabling instant alerts to occupants, authorities, or emergency responders.

1.5 Advantages Over Current System

The proposed system offers a more advanced and reliable approach to fire detection compared to existing solutions. By combining multi-sensor input with deep learning-based vision analysis, it significantly improves the accuracy and speed of fire detection while minimizing false alarms. The inclusion of object recognition and automatic fire class classification provides critical contextual awareness, enabling users to make safer and more effective decisions during emergencies. Additionally, the system's IoT connectivity ensures real-time communication and remote monitoring, making it suitable for a wide range of applications from residential and commercial settings to industrial and forest environments.

Chapter 2

LITERATURE SURVEY

A literature survey is a systematic review and synthesis of existing research, journal articles, and scholarly studies on a specific topic. It analyzes and compares previous work to understand the current state of knowledge, identify gaps, and relate existing findings to a research problem or objective.

2.1 Survey Papers

[1]. IoT Based Smart Home and Office Fire Notification Alert System

Author: Sunandita Debnath, Jashwin Gautam, Sahil Kumar Rai.

Date of publish of article: 2024 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)

Employs a network of temperature, smoke, and CO sensors integrated with an Arduino UNO that communicates via MQTT with a cloud platform to trigger alarms, sprinkler activation, and notifications upon abnormal readings. Provides real-time monitoring, remote control via a mobile application, and demonstrates high accuracy and responsiveness in extensive hardware testing. Relies on fixed threshold values that may not adapt well to varying conditions and lacks advanced machine learning for improved detection.

[2]. A Smart Fire Detector IoT System with Extinguisher Class Recommendation Using Deep Learning

Author: Tareq Khan.

Date of publish of article: IoT, 2023, Volume 4, Pages 558–581 (DOI: 10.3390/iot4040024)

Utilizes thermal and RGB cameras with the SSD-Inception-V2 deep learning model and homography transformation to detect fires and recommend the correct extinguisher class based on a lookup table. Enhances detection accuracy and provides tailored extinguisher recommendations, enabling rapid alerts and improved safety in indoor environments. Involves complex hardware and computationally intensive deep learning models that may increase costs and present alignment challenge.

[3]. Forest Fire Detection and Notification Method Based on AI and IoT Approaches

Author: Kuldoshbay Avazov, An Eui Hyun, Alabdulwahab Abrar Sami, Azizbek Khaitov, Akmalbek Bobomirzaevich Abdusalomov, and Young Im Cho

Date of publish of article: Future Internet, 2023, 15(2), 61 (DOI: 10.3390/fi15020061O)

Deploys IoT sensors to continuously monitor environmental parameters and applies deep learning algorithms to detect forest fires, subsequently notifying authorities via SMS and other communication channels. Improves detection accuracy, reduces false alarms, and facilitates rapid response in forested areas. Faces potential connectivity issues in remote regions and requires extensive, diverse training data for robust deep learning performance.

[4]. A Deep Learning Based Object Identification System for Forest Fire Detection

Author: Federico Guede-Fernández, Leonardo Martins, Rui Valente de Almeida, Hugo Gamboa, and Pedro Vieira.

Date of publish of article: Fire, 2021, 4(4), 75

Leverages convolutional neural networks to analyze forest imagery for automated object identification associated with fire incidents. Significantly enhances early detection accuracy and reduces false positives, thereby enabling prompt emergency responses. Its performance can be affected by environmental conditions and the high computational demands may limit real-time processing on low-power edge device.

2.2 Limitations of Existing Fire Detection System

- **Image-Only Deep Learning Models:** Deep learning–based forest fire detection systems rely mainly on camera images for identifying fire. If the visibility is poor due to fog, smoke density, night-time lighting, or low-resolution cameras, the system fails to detect the fire accurately. This leads to increased false positives and missed detections.
- **Lack of Visual Confirmation:** Many IoT-based fire alert systems used in homes and offices depend only on smoke, gas, or temperature sensors. These sensors trigger alarms based on threshold values but do not provide any image evidence. Users cannot confirm if the alert is real or false, which reduces the effectiveness of the system.
- **High Sensor False-Trigger Rate:** Conventional smart fire detectors often generate false alerts due to cooking smoke, dust particles, burning incense, or steam. Since these systems do not use AI-based identification, they cannot differentiate real fire from harmless

environmental changes, making the notification system less reliable.

- **High Computational Requirements:** Deep learning systems that classify fire and recommend extinguisher types require powerful processors or GPUs. Implementing such models on small IoT boards is difficult, and real-time processing becomes slow or impossible, limiting practical deployment

Chapter 3

METHODOLOGY

Studying the existing system helps understand current processes and identify requirements for developing a new system. This includes defining software, hardware, functional, and non-functional requirements needed for project implementation.

3.1 System Requirements

3.1.1 Functional Requirements

Functional requirements define the core operations the system must be capable of performing. In this project, the system must continuously collect data from temperature, gas, and smoke sensors and analyze these values to detect abnormal environmental conditions. When sensor thresholds are exceeded, the ESP32 camera must capture images and send them to the YOLOv8 model for real-time fire detection and object recognition. The system must classify the fire type based on detected objects and sensor inputs and automatically recommend an appropriate fire extinguisher class. It should transmit alerts and analysis results to a central dashboard, smartphone. These requirements ensure proper input handling, logical calculations, communication between IoT modules, and accurate machine-learning-driven decision making.

3.1.2 Non-Functional Requirements

A non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors. Non-functional requirements are often called qualities of a system.

- **Reliability:** The system provides continuous and stable monitoring, automatically reconnecting during network failures to prevent data loss. It ensures timely alerts whenever danger is detected, maintaining functionality even if a single sensor becomes temporarily unavailable.
- **Scalability:** The system is designed to support multiple ESP32 units and additional sensors without major changes. Its server and storage can easily handle increased data, making it suitable for larger environments and future expansion.
- **Performance:** The system processes sensor data in real time and transmits images within a few seconds, ensuring quick detection of hazards. Its optimized logic allows fast and accurate updates of the safe/danger status during continuous operation.

- **Usability:** The dashboard presents sensor values, images, and safety status in a clear and user-friendly interface. Alerts are simple to understand, and the system requires minimal setup, making it easy for anyone to operate and monitor.

3.1.3 Hardware Requirements

Processor	:	Intel i5 or equivalent
RAM	:	8GB or more
Storage	:	Minimum 4GB for image and data storage
Network	:	2.4GHz Wi-Fi connectivity
IoT Components	:	

- **ESP32-CAM Module:**

  Acts as the central controller of the system. It processes sensor data, captures images when danger thresholds are crossed, and sends images and readings to the backend server using Wi-Fi. Additionally, its compact size and low cost make it suitable for real-time IoT surveillance applications.

- **MQ-2 Smoke and Gas Sensor:**

  Detects smoke and flammable gases such as LPG and methane. Changes in gas concentration alter its output voltage, allowing early identification of fire or gas leakage. It offers fast response time, which helps in detecting hazardous conditions quickly.

- **DHT11 Temperature and Humidity Sensor:**

  Measures temperature and humidity to detect abnormal heat conditions. A sudden rise in temperature helps in early fire detection before visible smoke appears. It provides digital, calibrated output, making it easy to interface with the ESP32-CAM.

- **USB-to-TTL Converter:**

  Used during programming to upload code to the ESP32-CAM by enabling serial communication between the computer and the module. It also helps in debugging by allowing serial monitoring during development.

- **Power Supply Unit:**

  Provides a stable 5V power source required for continuous and reliable operation of the ESP32-CAM and sensors.

A standalone computer equipped with the required software should be placed in the monitoring area to receive, process, and store data from the IoT system. The camera module must be positioned properly in the environment to capture images during fire or smoke detection events.

3.1.4 Software Requirements

Operating System	:	Windows 10 and Android
Coding Language	:	Python 3.12, Embedded C, Flutter, Dart
IDE	:	VS Code, Arduino IDE, Android Studio
Libraries / Packages	:	Flask, OpenCV, TensorFlow, DHT library, WiFi library, Ultralytics, HTTP packages
Database	:	MongoDB
Application	:	Custom-developed mobile/desktop app for displaying alerts, images, and safety status

3.1.4.1 Ultralytics

Ultralytics provides a high-performance implementation of the YOLO (You Only Look Once) family of object detection models, including the latest YOLOv8. It is designed for real-time detection tasks, offering fast inference speeds, high accuracy, and an easy-to-use Python interface. The library includes pre-trained models, training utilities, and the tools for the dataset handling, making it suitable for both experimentation and production deployment. Ultralytics supports custom training, model optimization, and export to various formats like ONNX and TensorRT. Its modular design and detailed documentation make it ideal for modern computer vision applications such as traffic sign detection, face recognition, and autonomous navigation systems.

3.1.4.2 TensorFlow

TensorFlow is an open-source machine learning framework designed for building, training, and deploying deep learning models across a wide range of platforms. Developed to support large-scale numerical computation, TensorFlow provides a flexible architecture built around computational graphs, allowing developers to define complex neural networks with ease. It includes high-level APIs such as Keras for rapid model development and low-level operations for fine-grained control, making it accessible to both beginners and advanced researchers.

3.1.4.3 OpenCV

OpenCV (Open Source Computer Vision Library) is a widely-used open-source library for computer vision and machine learning tasks. Written in optimized C/C++ with Python and Java interfaces, OpenCV provides cross-platform compatibility and a modular structure that allows developers to include only the necessary components. The library offers over 2,500 optimized

algorithms covering a wide range of applications, from facial recognition and object detection and 3D reconstruction.

OpenCV is licensed under the BSD license, making it easy to use in commercial and research projects. With its extensive documentation, active community, and pre-built algorithms, OpenCV has become an essential tool for developers working on computer vision projects, enabling them to create sophisticated applications that can process images, detect objects, and track movements with ease.

3.1.4.4 Flask

Flask is a lightweight and extensible web framework for Python that follows a modular, "micro-framework" design. It provides essential tools such as URL routing, request handling, and template rendering while allowing developers to add only the components they need. Flask is widely used for building REST APIs, backend services, dashboards, and microservices because of its simplicity, flexibility, and ease of integration with databases, authentication systems, and third-party libraries. Its minimal structure makes it highly adaptable for both small prototypes and large-scale web applications.

3.1.4.5 DHT library

The DHT library is used for interacting with DHT-series digital temperature and humidity sensors. It handles the precise timing and communication required to read reliable sensor values, allowing developers to quickly access environmental data without complex low-level coding. The library supports multiple models in the DHT family and includes built-in error checking to improve data accuracy. It is commonly used in embedded systems, educational projects, and environmental monitoring applications.

3.1.4.6 Flask-CORS

Flask-CORS is an extension for the Flask web framework that enables Cross-Origin Resource Sharing (CORS) in web applications. It allows a server to safely handle requests from different domains, which is essential when frontend applications, mobile apps, or external systems need to interact with an API hosted elsewhere. Flask-CORS simplifies the configuration of CORS policies by providing decorators and global settings to control which origins, headers, and methods are permitted. This helps prevent browser security restrictions from blocking legitimate requests, making Flask-CORS widely used in modern web development, distributed systems, and API-based applications.

3.1.4.7 PyMongo

PyMongo is the official Python driver for interacting with MongoDB databases. It provides a simple and efficient interface for performing database operations such as inserting documents, running queries, updating records, and managing collections. PyMongo supports MongoDB's flexible document-oriented structure and offers features like indexing, aggregation pipelines, and connection pooling. It is widely used in applications that require fast, scalable, and schema-flexible data storage. With its straightforward API and strong integration with Python workflows, PyMongo makes working with NoSQL databases accessible and efficient for both small and large-scale applications.

3.1.4.8 Pillow

Pillow is a user-friendly and powerful Python imaging library that provides extensive tools for opening, manipulating, and saving image files. It supports a wide range of image formats and includes operations such as resizing, cropping, filtering, color adjustments, and format conversion. Pillow is commonly used in applications involving image processing, automation, digital content creation, and machine learning preprocessing. Its simplicity and broad feature set make it a popular choice for handling image-related tasks in Python, whether for basic editing or preparing data for advanced computer vision models.

3.2 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

3.2.1 Economic Feasibility

The proposed system is economically feasible because it uses low-cost IoT components such as the ESP32-CAM, MQ-2 sensor, and DHT11, all of which are inexpensive and widely available. The software stack, including Python, Flask, OpenCV, NumPy, and Arduino

libraries, is completely open-source, eliminating any need for paid software licenses. Since the backend can run on a regular standalone computer and the app is developed using freely available tools, the overall cost of development, deployment, and maintenance remains very low. This makes the system highly cost-effective for homes, institutions, and small organizations.

3.2.2 Technical Feasibility

The system is technically feasible because it is built using well-supported technologies and hardware that are easy to integrate. The ESP32-CAM provides built-in Wi-Fi and camera capability, making it suitable for real-time image capture and data transmission. Sensors such as MQ-2 and DHT are simple to interface and widely documented, ensuring reliable performance. The backend, developed using Python and Flask, offers a lightweight and scalable architecture for handling IoT communication. Since all components and libraries used are stable, mature, and compatible, the system can be implemented without requiring advanced or specialized technical infrastructure.

3.2.3 Social Feasibility

The project is socially feasible as it directly contributes to improving safety and awareness in households, offices, and institutions. By providing early fire detection, image alerts, and environmental monitoring, the system helps reduce risk and enhances public safety. It is easy to use and requires minimal technical knowledge, making it accessible to people of all age groups. The system also promotes the adoption of smart safety technologies, encouraging communities to shift toward proactive measures rather than reactive responses. Its low cost and usefulness make it socially acceptable and beneficial to society.

3.3 System Architecture

3.3.1 Architecture

The proposed framework integrates the ESP32-CAM for image capture and IoT sensors (MQ-2 and DHT) for environmental monitoring to detect early signs of fire or hazardous conditions in real time. The system overcomes the limitations of traditional fire alarms by combining camera-based visual monitoring, multi-sensor data analysis, and live alert generation through a custom user application. The architecture ensures continuous monitoring, fast data transmission, and immediate danger notification.

The proposed system works in a step-by-step process that starts with image and sensor data collection and ends with generating real-time alerts. The detailed workflow is outlined below:

1. Image and Sensor Capture

The ESP32-CAM continuously captures images of the environment, while the MQ-2 sensor measures smoke/gas levels and the DHT11/DHT22 sensor records temperature and humidity.

2. Data Packaging on ESP32-CAM

Each cycle, the ESP32-CAM bundles the latest image frame together with sensor readings into a structured data packet for processing.

3. Wireless Data Transmission

The ESP32-CAM sends the image and sensor data to the backend server through Wi-Fi using HTTP requests, ensuring real-time connectivity.

4. Server-Side Processing (Flask Backend)

The server receives the incoming data, stores it in the database, and analyzes sensor values against predefined thresholds to determine whether the environment is in a Safe or Danger state.

5. Alert Decision and Storage

If any sensor readings exceed danger limits, the backend generates an alert that includes the image, sensor values, and timestamp. This alert is saved for record keeping and monitoring.

6. App Interface Display

The processed information current status, captured image, and sensor readings is displayed in the custom user application, allowing the user to visually inspect the alert and understand the severity.

7. Continuous Monitoring

The entire process repeats continuously for every new cycle of image and sensor capture, ensuring uninterrupted monitoring of the environment and immediate detection of hazardous conditions.

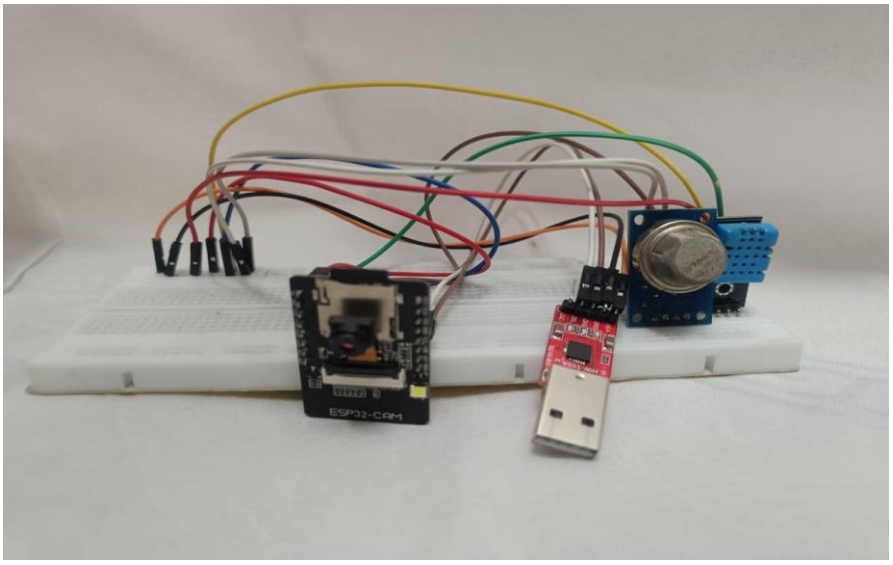

Fig. 3.1 Hardware Interconnection Diagram

All the hardware components work together to form an intelligent and responsive detection system. The sensors continuously measure environmental parameters, and the ESP32-CAM evaluates these readings to identify any sign of danger. If abnormal sensor values are detected, the ESP32-CAM captures an image and sends both the image and sensor data to the backend server. The server processes this information using a YOLOv8 deep learning model to confirm the presence of fire, identify the burning object, classify the fire type, and recommend the appropriate extinguisher. The final result is then displayed to the user through the mobile application, ensuring timely alert generation and helping users take immediate action.

3.3.2 Data Acquisition

This step involves collecting environmental and visual data using IoT devices.

- **Temperature Sensor (DHT11)** continuously monitors ambient heat levels.
- **Gas/Smoke Sensor (MQ-2)** detects smoke density and combustible gases.
- **Camera Module (ESP32-CAM)** captures real-time images when sensor values cross predefined thresholds.

The combination of sensor readings and captured images ensures that both physical and visual cues of fire are monitored.

3.3.3 Fire Detection Using Sensors

This stage, raw sensor data is analysed to determine abnormal patterns indicative of a Fire.

- The system compares temperature(35°C) and gas levels with predefined safety limits.
- When values exceed these thresholds, a preliminary alert is generated, triggering the camera module and forwarding data for further analysis.

This early detection mechanism helps minimize false positives.

3.3.4 Image-Based Fire and Object Detection (YOLOv8)

This step performs visual fire confirmation and object recognition using a YOLOv8 deep learning model.

- The model processes captured images to detect fire regions with high accuracy.
- It identifies objects involved in the fire (e.g., electrical appliances, wood, oil containers).
- Using object information, the system categorizes the fire type.

This object-aware classification strengthens the reliability of fire identification in complex environments.

3.3.5 Fire Classification and Extinguisher Recommendation

Once object detection is completed, the system determines the fire class:

- **Class A:** Combustibles like wood, paper, cloth
- **Class B:** Flammable liquids
- **Class C:** Electrical equipment
- **Class D:** Metals
- **Class K:** Cooking oils/fats

Based on the detected fire class, the system automatically recommends the appropriate extinguisher type (water, CO_2, foam, drypowder, etc.).

This ensures correct and safe suppression measures.

Table 3.1 Recommended extinguishing agents for various classes of fire

Fire Class/Extinguishers	Water	Water Mist	Foam	ABC Dry Powder	CO_2	Wet Chemical	Specialist Powder
Class A	√	√	√	√			
Class B		√	√	√	√	√	
Class C		√		√			
Class D							√
Class K		√		√	√		
Class F							√

3.3.6 Alert Generation and Notification

After confirming the fire event, the system instantly compiles key details such as:

- Fire location
- Sensor readings
- Detected objects
- Fire class and severity
- Extinguisher recommendation

These details are transmitted to a **central server** and delivered as real-time alerts to users or emergency teams through mobile apps or dashboards. This can be seen in Fig 3.2. This enables rapid action and reduces risk.

3.3.7 Logging and Monitoring

All sensor data, detection results, and alert logs are stored for:

- Trend analysis
- System performance evaluation
- Future model improvements

This ensures continuous learning and long-term reliability of the system.

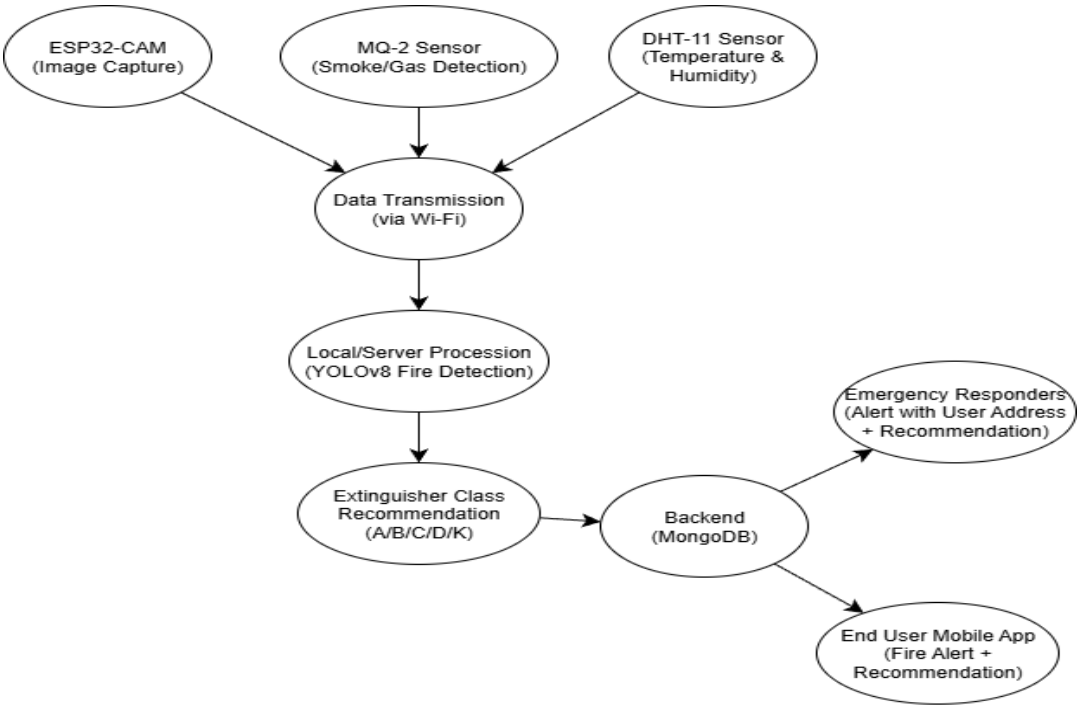

Fig. 3.2 System Architecture

Chapter 4

IMPLEMENTATION

The goal of the programming or implementation phase is to translate the design of the system produced during the design phase into code in a given programming language, which can be executed by the computer and that performs the computation specified by the design. Implementation of any software is always preceded by important decisions regarding selection of the platform, the language used etc., these decisions are often influenced by several factors such as real environment in which the system works, the speed that is required, the security concerns, other implementation specific details etc.,

4.1 Data Preparation

The ClassesOfFire Dataset forms the foundation for the development and evaluation of the proposed fire classification model. The dataset contains a total of 4,481 images categorized into six major fire classes following standard fire classification guidelines. These classes include Class A, Class B, Class C, Class D, Class K, and No Fire. The images represent a wide range of real-world fire scenarios, including ordinary combustible material fires, flammable gas and liquid fires, oil-based fires, combustible metal and electrical fires, cooking oil fires, and non-fire conditions. This diversity makes the dataset suitable for training deep-learning models aimed at accurate fire detection across various environments.

The dataset is sourced from multiple publicly available repositories and captures rich variations in lighting, background clutter, viewing angles, fire intensity, and scene complexity. Such real-world inconsistencies contribute positively to model robustness but also introduce challenges such as varying resolutions and visual noise. A notable limitation of the dataset is its significant class imbalance. Classes such as Class A and No Fire contain substantially larger numbers of samples compared to Classes B, D, and K, which have comparatively fewer images. This imbalance can lead to biased training, where the model becomes more sensitive to majority classes while underperforming on minority classes.

To address this issue, additional images were collected through web-scraping techniques to improve the distribution across underrepresented classes. Tools such as BeautifulSoup and Bing Image Downloader were utilized. The scraped images were manually reviewed, filtered for relevance, and integrated into the corresponding classes to achieve a more balanced dataset. This augmentation step enhances class-level uniformity and improves the model's generalization capability.

Table 4.1: Dataset Overview

Fire Class	Description	Image Count
Class A	Ordinary combustible materials (wood, paper, cloth, etc.)	**2,446**
Class B	Flammable gas and chemical liquid fires	**186**
Class C	Oil-based fires	**462**
Class D	Combustible metal and electrical fires	**169**
Class K	Cooking oil and fat fires	**164**
No Fire	Non-fire conditions and false-positive scenarios	**1,054**
Total	—	**4,481**

4.1.1 Database Organization

The dataset is organized using a class-wise folder structure, where each folder corresponds to one of the six fire categories. These folder names directly represent the class labels used for the image classification task. YOLOv8-Cls only expects images to be placed inside folders named after their respective classes. During training, YOLOv8 automatically assigns numerical class IDs based on the folder order. Therefore, manual mapping of class names to numeric identifiers is not necessary. Each fire category is internally mapped to an index (0 to 5) by the framework without requiring user intervention.

To prepare the dataset for training, the images were divided into train, validation, and test subsets. All splits maintained the same folder-based class structure to ensure seamless integration with YOLOv8's classification data loader. Care was taken to preserve the relative class proportions while mitigating the impact of class imbalance.This directory-style organization makes the dataset clean, modular, and directly compatible with YOLOv8-Cls, enabling efficient loading, preprocessing, and training without additional label files or annotation formats.

4.2 Model Training

4.2.1 Choosing YOLOv8s Model

- Balanced accuracy and speed, better than YOLOv8n for real-time detection.
- Lightweight and suitable for edge devices like IoT hardware.
- Efficient for GPU and moderate CPU training, requiring less memory than

YOLOv8m.

- Performs well in detecting fire and smoke in various indoor and outdoor scenarios, making it ideal for IoT-based fire extinguisher class recommendation systems.

4.2.2 Training Environment

- Google Colab GPU (T4)
- Python 3.12
- Ultralytics YOLOv8 framework

4.2.3 Training Procedure

The fire-type classification model was developed using the YOLOv8s-cls architecture to categorize 33 different fire and non-fire classes. The dataset consisted of real-world images representing diverse fire scenarios such as electrical fires, oil fires, metal fires, household fires, chemical fire reactions, and a separate "No Fire'' category. To ensure proper training and evaluation, the dataset was organized into class-wise folders and automatically split into training (80%) and validation (20%) sets using a stratified splitting approach. In total, 3603 images were allocated for training and 917 images for validation.

After organizing the dataset, the YOLOv8s-cls model was initialized with pretrained weights to accelerate convergence and improve accuracy on complex fire textures. The model was trained for 25 epochs with an image size of 224×224, where Automatic Mixed Precision (AMP) was enabled to optimize GPU utilization. During training, the model progressively reduced its loss from 3.15 in the first epoch to around 0.44 by the final epoch, indicating strong feature learning and stable gradient updates.

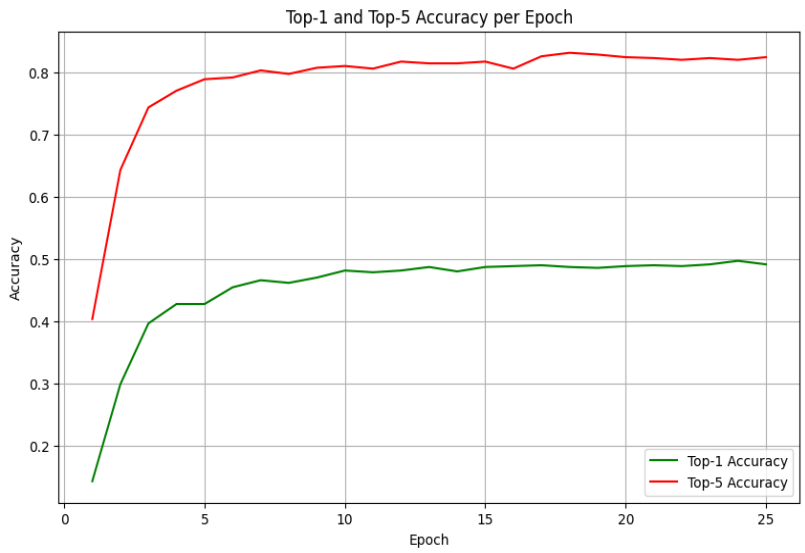

Fig. 4.1 Top-1 and Top-5 Accuracy per Epoch

The validation metrics further confirmed the model's performance. The Fig. 4.1 illustrates Top-1 accuracy reached 61.06%, while the Top-5 accuracy achieved 85.93%, showing that the model is able to correctly identify the exact fire class in most cases and list the correct class within its top five predictions with high reliability. The model demonstrated fast processing speeds, averaging around 0.45 ms per image during inference, making it suitable for real-time IoT deployments. The evaluation also generated a detailed confusion matrix, which helps analyze misclassifications between visually similar fire types such as chemical flames, oil flames, and burning materials. A sample test prediction was run on a "No Fire'' image, validating that the model could successfully perform inference on real input data.

The classification pipeline successfully transformed a raw multi-class fire dataset into a fully trained deep learning model capable of recognizing different fire scenarios. The use of YOLOv8s-cls provided a lightweight yet powerful backbone, balancing speed and accuracy, which is ideal for IoT-based fire detection and fire-extinguisher class recommendation systems. The training procedure, metrics, and exported model collectively demonstrate the effectiveness of the approach for intelligent safety applications.

4.2.4 Final Trained Model Output

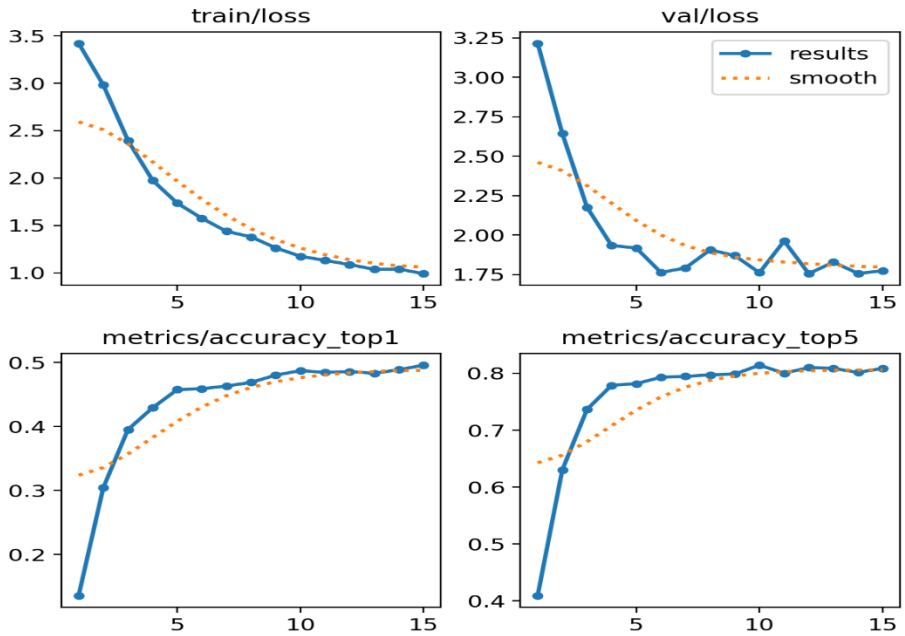

Fig. 4.2 Evaluation Metrics Graphs

The final trained model from the Fig 4.2 demonstrates reliable performance for multi-class fire classification using the YOLOv8s-cls architecture. After completing 25 epochs of training, the model achieved a Top-1 accuracy of 0.6107, indicating that in approximately 61% of the validation samples, the model's highest-scoring prediction exactly matched the correct fire

class. The Top-5 accuracy reached 0.8593, meaning the correct label appeared within the top five predictions for nearly 86% of validation images. This high Top-5 accuracy is particularly valuable because the system uses a decision strategy where the model extracts the top five predicted classes, checks how many of them belong to the same fire category, and then selects the final fire class based on this majority logic. This approach enhances stability and reduces misclassification among visually similar fire types. The model also achieved a fitness score of 0.7350, reflecting a balanced overall performance combining both accuracy metrics.

4.3 Fire Detection

4.3.1 Sensor-Based Fire Detection Module

The fire detection module operates through a multi-layered approach that combines IoT sensor data with deep learning-based image analysis. The ESP32-CAM continuously monitors environmental conditions through integrated sensors while simultaneously capturing visual data for analysis.

Environmental Monitoring

The system employs three primary sensors for environmental monitoring:

DHT11 Temperature and Humidity Sensor:

- Monitors ambient temperature with a range of 0°C to 50°C.
- Detects sudden temperature spikes that may indicate fire.
- Threshold temperature set at 35°C for alert generation.
- Provides humidity readings to distinguish between actual fire conditions and steam/moisture.

MQ-2 Gas and Smoke Sensor:

- Detects combustible gases including LPG, propane, methane, and hydrogen.
- Measures smoke density through changes in sensor resistance.
- Analog output voltage varies proportionally with gas concentration.
- Calibrated to trigger alerts when smoke/gas levels exceed safe thresholds.

ESP32-CAM Image Capture:

- Captures real-time images at 640×480 resolution.
- Automatically triggered when sensor thresholds are exceeded.
- Provides visual evidence for deep learning analysis.
- Enables remote monitoring through image transmission.

4.3.2 YOLOv8-Based Fire Detection

Fire Region Detection:

Once the sensor thresholds are exceeded and an image is captured, the YOLOv8 deep learning model analyzes the image to detect regions containing fire or flames. The model is trained on diverse fire

scenarios, enabling it to accurately recognize flame patterns, smoke characteristics, and other fire-related visual features under different environmental conditions. This training allows the system to distinguish fire from non-fire elements with high reliability. As a result, the model achieves a high average detection confidence of 87.59% for fire presence. Additionally, the image processing time is approximately 0.45 ms, which ensures rapid analysis and supports real-time fire detection and immediate system response.

Object Recognition in Fire Scenes:

In addition to detecting fire regions, the YOLOv8 model performs object recognition within the identified fire areas. It identifies burning objects such as electrical appliances, furniture, and cooking equipment, which helps in understanding the nature of the fire. The model is capable of recognizing 33 different fire and non-fire classes, providing detailed contextual information about the fire scene. This object-level understanding enables accurate fire classification based on the type of material involved, which is essential for assessing fire severity and supporting appropriate fire response and extinguisher class recommendations.

4.4 Classification & Recommendation

4.4.1 Fire Classification System

The fire classification module determines the specific type of fire based on the burning object identified by YOLOv8. This classification follows standardized fire safety categories established by the National Fire Protection Association (NFPA).

Fire Class Categories

Class A - Ordinary Combustibles

Class A fires involve ordinary combustible materials such as wood, paper, cloth, plastics, and rubber. These fires are characterized by their tendency to leave ash residue and can typically be extinguished using water-based agents. The YOLOv8 model identifies Class A fires by detecting smoke patterns, flame colors ranging from yellow to orange, and the texture of burning materials. Common scenarios include household fires involving furniture, office fires with paper and documents, and outdoor fires involving vegetation or wooden structures.

Class B - Flammable Liquids and Gases

Class B fires involve flammable liquids and gases including gasoline, oil, grease, paint, alcohols, and propane. These fires are characterized by their liquid fuel source which allows them to spread rapidly and produce intense heat. Detection indicators include the presence of liquid surfaces on fire, chemical containers, and fuel storage areas. The model recognizes these fires through visual patterns associated with liquid-based combustion and the characteristic blue-orange flames produced by hydrocarbon fuels.

Class C - Electrical Fires

Class C fires involve energized electrical equipment including wiring, circuit breakers, and electrical appliances. The primary characteristic distinguishing these fires is the presence of live electrical current, which creates a significant risk of electrocution during suppression. The system identifies Class C fires by detecting electrical devices, power outlets, and electronic equipment in the fire region. Visual indicators include sparking, melting of electrical components, and fires originating from power sources or electrical panels.

Class D - Combustible Metals

Class D fires involve combustible metals such as magnesium, titanium, sodium, and lithium. These fires are characterized by extremely high temperatures and violent reactions with water or standard extinguishing agents. Detection indicators include the presence of metal components, industrial settings, and bright white or bluish white flames. The YOLOv8 model is trained to recognize metallic surfaces and industrial equipment that may indicate the presence of combustible metals.

Class K - Kitchen/Cooking Fires

Class K fires specifically involve cooking oils, animal fats, and vegetable oils commonly found in commercial and residential kitchens. These fires are characterized by high-temperature oils that can reignite easily and pose unique suppression challenges. The system identifies Class K fires by detecting cooking equipment such as stoves, fryers, and oil containers. Visual patterns include grease fires with characteristic yellow-orange flames and smoke, along with the presence of kitchen appliances and cookware.

4.4.2 Classification Algorithm

The system employs a majority-voting mechanism for robust fire classification:

```
def classify_fire(detected_objects, top5_predictions):
  # Extract top 5 predicted classes from YOLOv8
  predictions = model.predict_top5(image)
    # Map detected objects to fire classes
  fire_class_votes = {
    'Class A': 0,
    'Class B': 0,
    'Class C': 0,
    'Class D': 0,
    'Class K': 0
  }
    # Count votes for each fire class
```

```
    for prediction in predictions:
        fire_class = map_object_to_fire_class(prediction)
        fire_class_votes[fire_class] += 1
        # Select class with maximum votes
    final_fire_class = max(fire_class_votes, key=fire_class_votes.get)
    confidence = calculate_confidence(fire_class_votes)
        return final_fire_class, confidence
```

This approach ensures that even if one or two predictions are uncertain, the overall classification remains accurate by considering multiple predictions and selecting the most frequently occurring fire class.

4.4.3 Extinguisher Recommendation Engine

The Extinguisher Recommendation Engine is responsible for suggesting the most appropriate fire extinguisher based on the identified fire class. Once the fire detection and classification module determines the type of fire, this engine applies predefined rules to ensure that the recommended extinguisher is both safe and effective for controlling the fire.

Different fire classes require different extinguishing agents due to the nature of the burning material. The recommendation logic is designed to prevent the use of incorrect extinguishers, which may worsen the situation or cause safety hazards.

Table 4.2 Mapping of Fire Classes to Suitable Extinguishers

Fire Class	Recommended Extinguisher	Operational Basis
Class A	Water, Foam, Multi-purpose Dry Chemical	These extinguishers cool the burning material and remove heat, making them effective for fires involving solid combustible materials such as wood and paper.
Class B	CO_2, Dry Chemical, Foam	These agents smother the flames by cutting off the oxygen supply, making them suitable for flammable liquid fires.
Class C	CO_2, Dry Chemical (non-conductive)	Non-conductive extinguishing agents are used to safely suppress fires involving electrical equipment without causing electric shock.
Class D	Dry Powder (specialized metal fire extinguisher)	These extinguishers are specifically designed to handle combustible metal fires and prevent chemical reactions with metals.

Class K	Wet Chemical	Wet chemical extinguishers create a protective barrier between hot oils and oxygen, making them ideal for kitchen-related fires.

4.4.4 Real-Time Processing Pipeline

The complete classification and recommendation pipeline operates as follows:

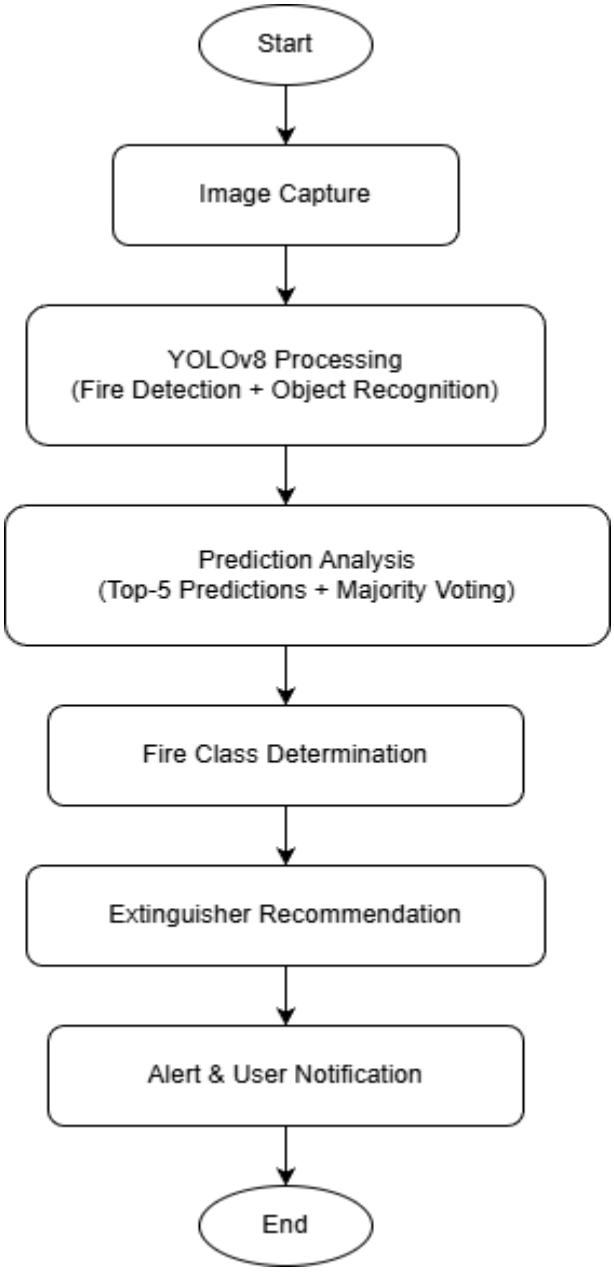

Fig 4.3 System Flow

This integrated approach ensures that users receive not only timely fire alerts but also actionable guidance on the most effective suppression method, significantly improving fire safety response and outcomes. The Fig 4.3 represents the working process of the YOLOv8-based fire detection system integrated with an intelligent extinguisher recommendation and alert mechanism. The process begins

with image capture triggered after fire-related sensor thresholds are exceeded. The captured image is then processed using the YOLOv8 deep learning model, which performs fire detection and object recognition simultaneously. The system analyzes the top-5 prediction outputs and applies majority voting to improve detection reliability. Based on the final prediction, the fire class is determined according to the type of burning material. Once the fire class is identified, the appropriate fire extinguisher is mapped and recommended. Finally, an alert is transmitted and the user is notified in real time, ensuring quick and effective response to fire incidents.

4.5 System Testing

System testing is a crucial phase that evaluates the complete and integrated Fire Detection and Alert System to ensure that the entire application operates correctly under various conditions. This process verifies that the system not only meets its functional requirements but also performs reliably, efficiently, and consistently when all modules mobile frontend, backend APIs, YOLOv8 detection engine, authentication components, and email alert mechanism work together as a whole. The testing procedures carried out include unit testing, integration testing, validation testing, output testing, system testing, white box testing, and black box testing. Each stage ensured that the system was thoroughly examined before deployment.

4.5.1 Unit Testing

Unit testing examined the smallest components of the system to ensure that each module behaved correctly before integration. Individual functions, classes, and logical operations were tested in isolation to confirm accuracy and stability. Unit testing focuses on validating individual components, such as functions and services, in isolation.

Executed Tests

- **Configuration Loading:** Ensured that the application correctly loads the testing configuration.
- **YOLO Model Initialization**: Verified proper loading and initialization of the YOLO object detection model.

Table 4.3 Unit Test Result

Test Case	Component	Status
Config Load	App Config	Passed
Model Load	YOLO Service	Passed

4.5.2 Integration Testing

Integration testing validates that modules communicate and interact correctly.

Executed Tests

- **Prediction Endpoint**: Tested the /predict API by mocking both the YOLO service and database. Confirmed correct handling of image input, prediction processing, logging, and response generation.
- **Database Interaction**: Verified retrieval of user data and insertion of prediction logs during the prediction pipeline.

Table 4.4 Integration Test result

Test Case	Interaction	Status
Predict API	API → YOLO Service → Database	Passed

4.5.3 Validation Testing

Validation testing ensures that the implemented system meets the functional requirements.

Feature Checklist

- **Real-time Detection**: Validated through PredictFireClassScreen capturing and sending images for analysis.
- **User Authentication**: Confirmed proper workflow including OTP generation and password reset in ForgotPasswordScreen.
- **Alert System**: Verified email alert functionality with simulated GPS data when fire is detected

Status

All core features were verified and validated against the system requirements.

4.5.4 Output Testing

Output testing verifies that the system produces correct and meaningful results for the inputs provided. It ensures that the final output is accurate and user-friendly. This verifies that the system outputs correct formats and data structures.

Executed Tests

- JSON Structure Validation: Ensured /predict response contains all required fields (success, predicted_class, predicted_confidence, recommendation).
- Data Type Verification: Confirmed correct value types (e.g., confidence as float, predicted class as string).

Table 4.5 Output Testing Result

Output	Expected Format	Status
Prediction Response	JSON containing class & confidence	Passed

4.5.5 System Testing

System testing evaluates the complete, integrated system to verify compliance with both functional and non-functional requirements. It checks the system as a whole.System testing evaluates the entire integrated system from end to end.

1. **User Authentication** – User logs in via Auth Routes.
2. **Image Acquisition** – Captured via camera or gallery (validated in PredictFireClassScreen).
3. **Backend Processing** – YOLOv8 model receives and analyzes the input image.
4. **Response Handling** – System returns a prediction (Fire or Normal).
5. **Alert Triggering** – If fire is detected, email alert is sent and UI displays a warning.

The full system flow was validated successfully, confirming that all integrated components operate cohesively.

4.5.6 White Box Testing

White box testing focuses on verifying internal code logic and structure.

Analyzed Modules

- **prediction_routes.py**
  - Validates image file presence.
  - Executes conditional alert dispatch logic.
  - Includes robust exception handling.
- **forgot_password_screen.dart**
  - Uses setState for UI state management.
  - Validates form inputs before API interaction.

4.5.7 Black Box Testing

Black box testing evaluates the system purely based on inputs and outputs without inspecting its internal code. It focuses on functional correctness from the user's perspective. Black box testing evaluates system behaviour without internal knowledge.

Table 4.6 Functional Scenarios

Scenario	Input	Expected Outcome	Actual Result
Valid Image Upload	Fire image	Predict “Fire”	Correct prediction via API
Missing File	No image provided	400 Bad Request	Returns {"error": "No image file provided"}
Forgot Password	Registered email	OTP sent	OTP triggered via email service

The system has successfully passed all levels of testing, including Unit, Integration, Validation, Output, System, White-box, and Black-box testing. This comprehensive test coverage confirms that every component of the application functions correctly both independently and as part of the integrated system. The results demonstrate that the software meets all specified requirements, performs reliably under expected conditions, and is ready for real-world deployment with a high level of confidence in its stability, accuracy, and usability.

Chapter 5

RESULTS

The proposed hybrid fire detection system integrating IoT sensors with YOLOv8-based image detection was experimentally validated for accuracy, reliability, and real-time performance. The system effectively combined temperature and smoke sensor data with visual analysis to detect and classify fire incidents with minimal false alarms. The integration enabled early warning through sensors and visual confirmation through YOLOv8, ensuring robust and precise detection across diverse environments. The classification module accurately identified the type of fire like ordinary combustibles, flammable liquids, or electrical fires by correlating sensor readings with image-based features. This facilitated the automatic recommendation of suitable fire extinguishers, thereby enhancing the overall response efficiency and safety. The alert system successfully transmitted notifications to connected devices such as smartphones and web dashboards, containing key information including fire location, sensor readings, YOLOv8 confidence scores, and extinguisher recommendations. Continuous data logging supported system evaluation, enabling performance monitoring and further optimization. The experimental results demonstrated high detection accuracy, fast response, and strong reliability under real-world conditions in residential, commercial, and industrial scenarios.

5.1 Comparative Study

Table 5.1 Model Results

Metric	Detection Accuracy	Mean False Positive Rate	Response Time	Classification Accuracy	YOLOv8 Detection Confidence
Achieved Value	91.02%	1.58%	399 ms	86.17%	87.59%
Description	Percentage of actual fire events correctly detected by sensors and YOLOv8.	Percentage of non-fire events incorrectly classified as fire.	Time between fire occurrence and alert generation.	Accuracy in correctly identifying fire type (combustible, liquid, electrical).	Minimum confidence score for fire region detection in images.

These metrics collectively provide a strong analytical foundation for evaluating the reliability and real-time performance of the proposed fire detection system. They also offer clear benchmarks that can be used for further optimization, comparative studies, and validation against existing detection models. By maintaining consistent performance across these

indicators, the system demonstrates its suitability for practical deployment in real-world monitoring environments.

5.2 Model Evaluation

The YOLO image classification model was evaluated to assess its ability to accurately identify and categorize fire-related images within the dataset. Since YOLO follows a single-stage architecture, it performs feature extraction and classification in a unified pipeline, allowing fast and efficient processing of real-time inputs. In this evaluation phase, the model's performance was measured using standard classification and detection metrics, enabling a comprehensive understanding of its accuracy, reliability, and consistency. These results help determine whether the trained model is suitable for deployment and highlight areas where further tuning or dataset refinement may be required.

5.2.1 Confusion Matrix

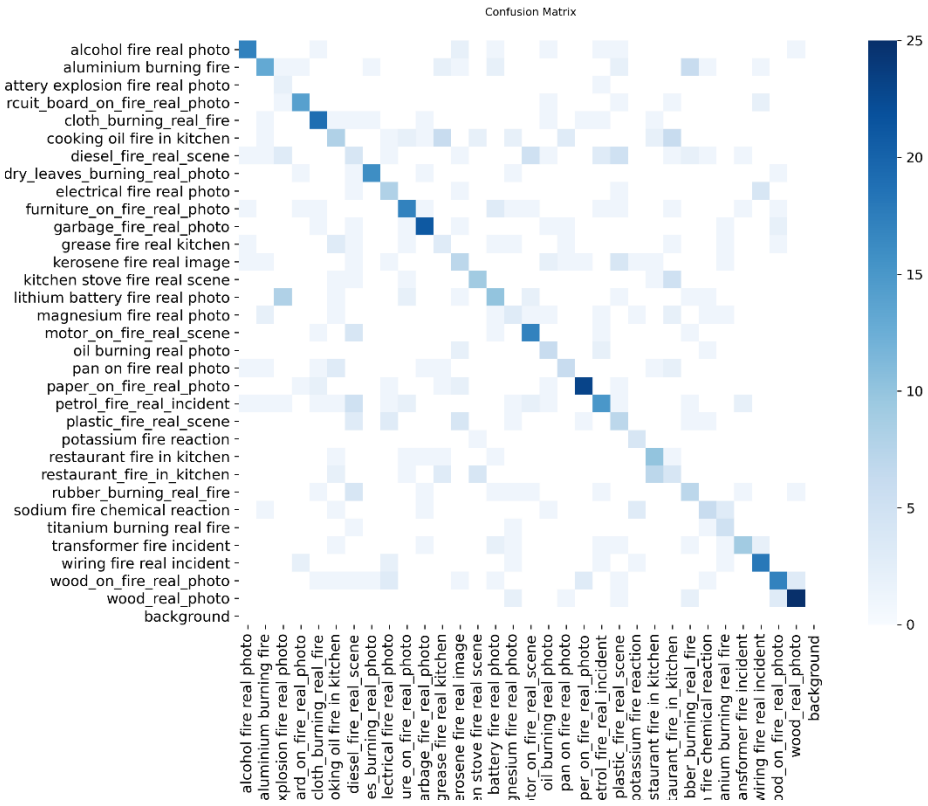

Fig. 5.1 Confusion Matrix

The confusion matrix provides an overall view of how well the YOLO classification model distinguishes between different fire categories. Fig 5.1 shows the Confusion Matrix of the trained model. Most predictions align along the diagonal, indicating that the model correctly identifies a large proportion of samples across the dataset. Misclassifications appear as lighter off-diagonal cells, suggesting that a few visually similar fire types—such as kitchen fire, oil

fire, or electrical fire—tend to overlap due to shared colour patterns or flame structures. These errors are expected in multi-class fire datasets where inter-class visual differences can be subtle. The distribution of values across the matrix also reveals the presence of class imbalance, where some categories have fewer samples and thus exhibit lower prediction confidence. Overall, the confusion matrix helps identify which classes require additional augmentation, improved annotation quality, or hyperparameter tuning to further enhance classification accuracy.

5.2.2 Validation Batch – Ground-Truth Labels

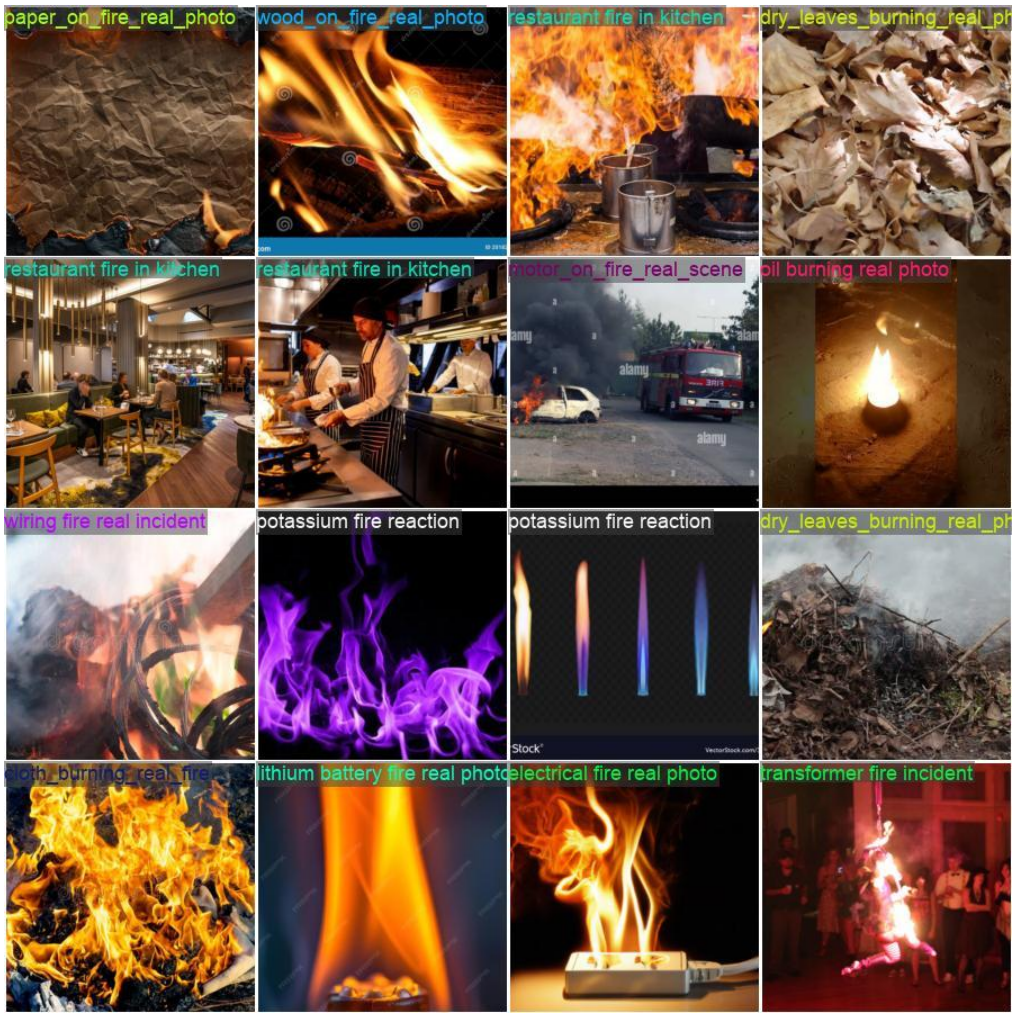

Fig. 5.2 True Label Images

The validation batch labelled images showcase the actual ground-truth categories assigned during dataset preparation. This Fig 5.2 highlights the diversity of the dataset, including household fires, industrial fires, chemical reactions, vehicle fires, and natural material combustion. Such variability ensures that the model learns a broad spectrum of flame characteristics, smoke behaviors, and environmental contexts. Observing the ground-truth batch also allows for manual inspection of dataset quality, helping confirm that annotations are accurate, consistently formatted, and reflective of real-world fire scenarios. This step is

essential to verify that the evaluation process is based on reliable and representative data.

5.2.3 Validation Batch – Model Predictions

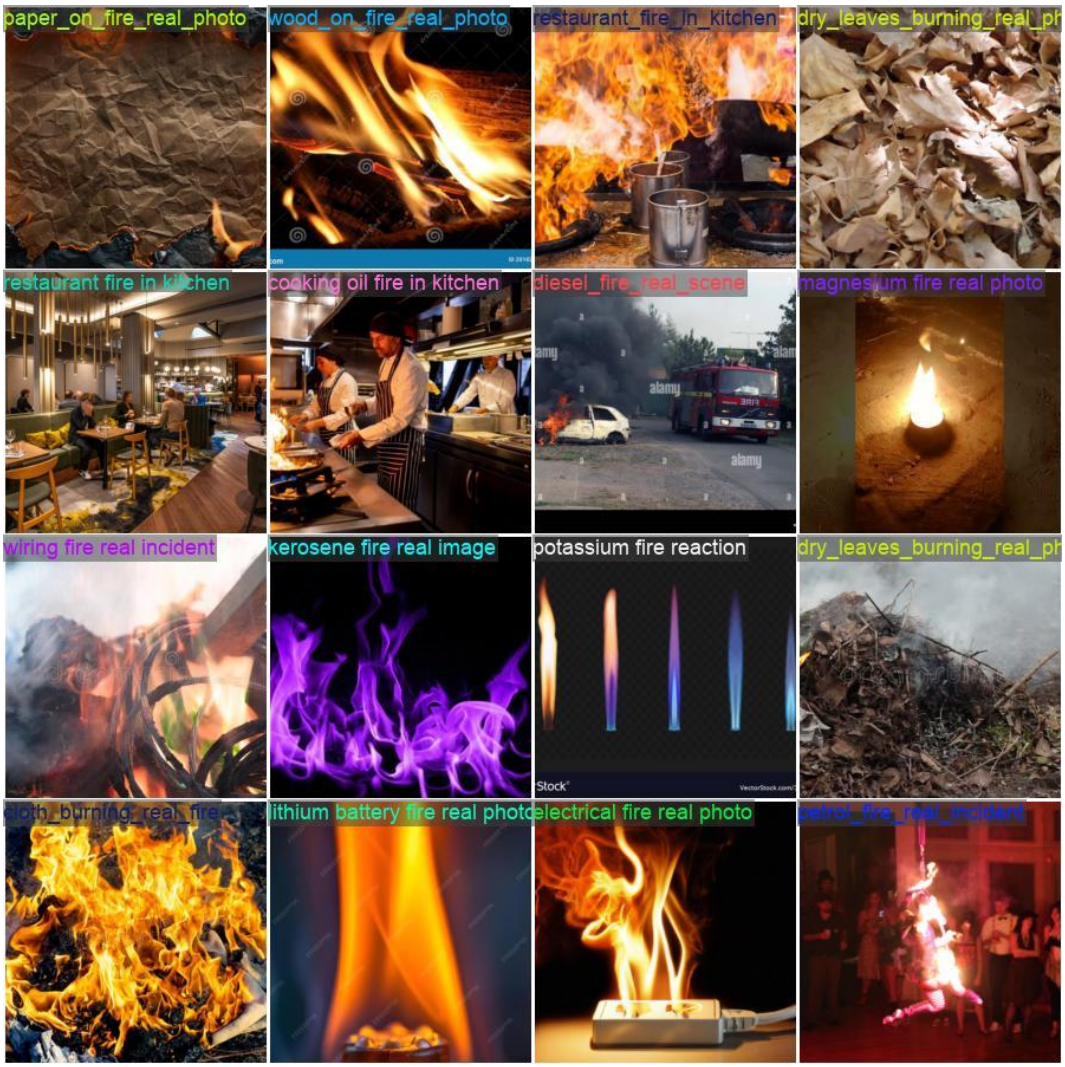

Fig. 5.3 Predicted Images

The prediction batch displays how the trained YOLO model interprets the same set of validation images. Comparing these predictions with ground-truth labels offers insight into the model's real-world behaviour and classification stability. As seen in the Fig 5.3 Many images are correctly labelled, demonstrating that the model has effectively learned distinguishing features for most fire categories. However, the predicted outputs also reveal cases where the model shows uncertainty, such as confusing two fire types with similar visual textures or flame patterns. These observations, combined with numerical evaluation, provide a strong basis for identifying potential improvements, including fine-tuning model weights, adjusting confidence thresholds, or enriching the dataset with additional high-quality samples. Overall, the predicted batch analysis complements the confusion matrices by providing a qualitative assessment of model performance.

5.3 Snapshots

The snapshots included in this section illustrate the real-time working of the proposed system. They show how the sensors detect abnormal conditions, how images are captured using the ESP32-CAM, and how the classification results and alerts are displayed in the application. These snapshots help in understanding the practical behavior of the system and confirm that the implemented modules function correctly.

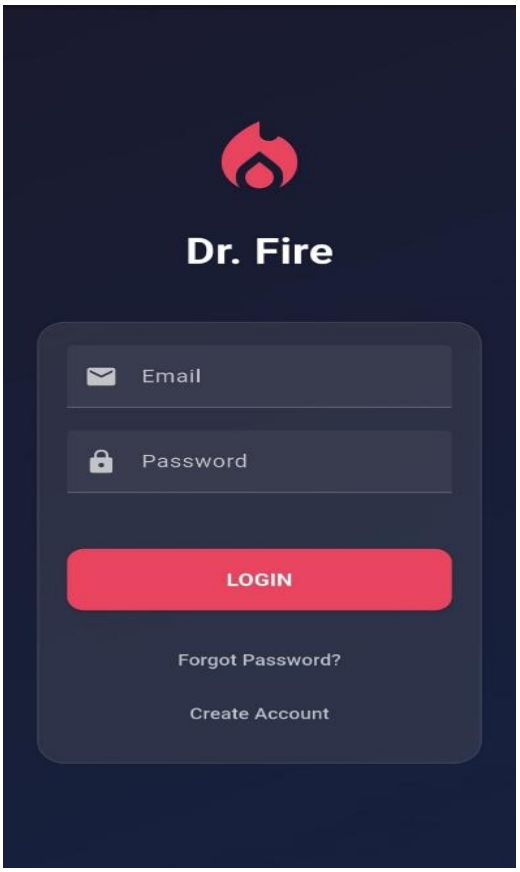

Fig. 5.4 Login Screen

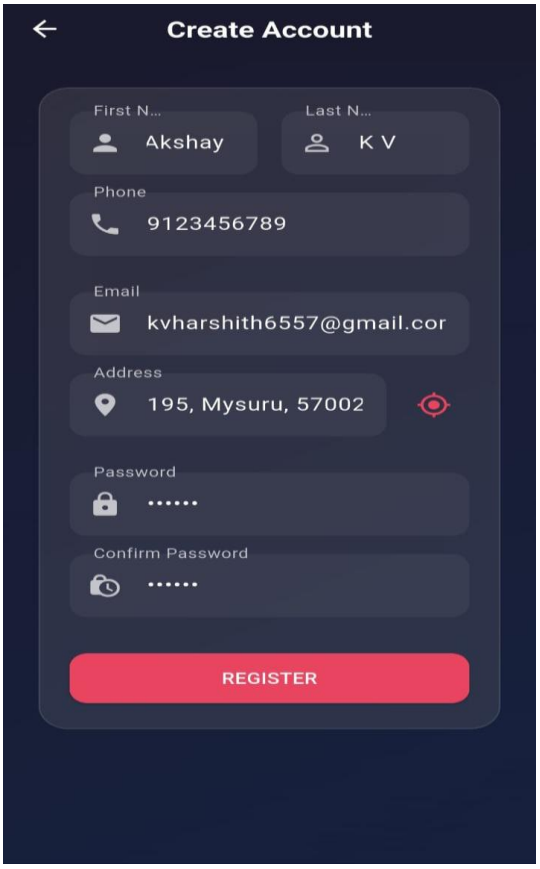

Fig. 5.5 Create Account Screen

1. Login Screen

Fig. 5.4 represents the user authentication interface of the Dr. Fire application. Users enter their email and password to securely access the system. The simple and minimal UI ensures easy and quick login.

2. Create Account Screen

Fig. 5.5 allows new users to register by entering their personal details such as name, phone number, email, address, and password. It ensures proper onboarding and secure profile creation within the application. The design maintains clarity and ease of input.

Fig. 5.6 System Safe Dashboard

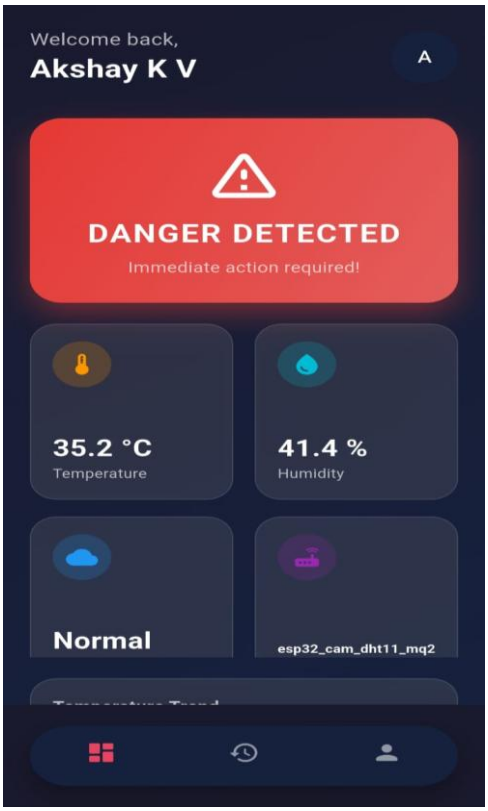

Fig. 5.7 Danger Detected Dashboard

3. System Safe Dashboard

Fig. 5.6 represents real-time sensor readings when the environment is safe. Temperature, humidity, and gas status are shown in a clear card layout for quick monitoring. The green "System Safe" banner indicates that all sensors are functioning normally.

4. Danger Detected Dashboard

Fig. 5.7 illustrates the screen triggered when abnormal sensor readings indicate a potential fire hazard. The red "Danger Detected" alert warns the user to take immediate action. Updated temperature and humidity values help assess the severity of the situation.

5. Incident History Screen

Fig. 5.8 displays previously recorded fire incidents along with the exact date and time. It shows sensor readings like temperature and humidity at the moment of detection, along with the predicted fire class. This helps users review and understand past alerts for safety analysis.

6. My Profile Screen

Fig. 5.9 shows the user's personal details such as name, email, phone number, address, and role. It allows users to easily manage their profile information and access the logout option. The clean layout ensures quick navigation and account visibility.

Fig. 5.8 Incident History Screen

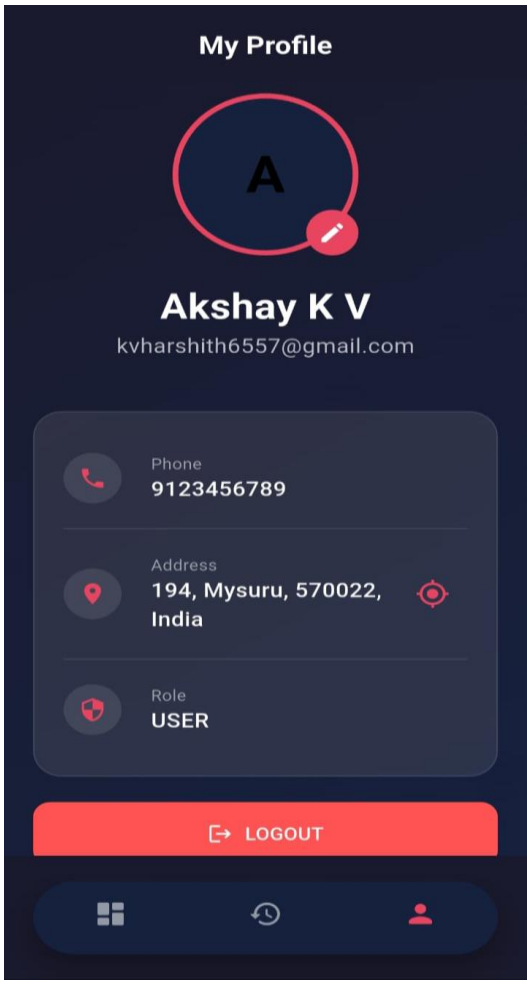

Fig. 5.9 My Profile Screen

7. Fire Analysis (Upload Screen)

Fig. 5.10 allows users to upload an image via the camera or gallery for fire classification. The interface guides the user to select or capture an image and then analyze it using the onboard model. It serves as the entry point for image-based fire detection.

8. Fire Analysis (Result Screen)

Fig. 5.11 allows the user uploads an image, the system classifies the fire and displays the predicted fire class along with confidence percentage. It also provides extinguisher recommendations appropriate for the detected fire type. An alert notification confirms that safety emails have been successfully sent.

9. Emergency Alert Email Notification

Fig. 5.12 feature automatically sends a detailed alert email to registered emergency responders whenever a fire hazard is detected. The notification includes the fire class, sensor readings, timestamp, and user location to enable quick response. This ensures timely action and enhances the overall safety and reliability of the system.

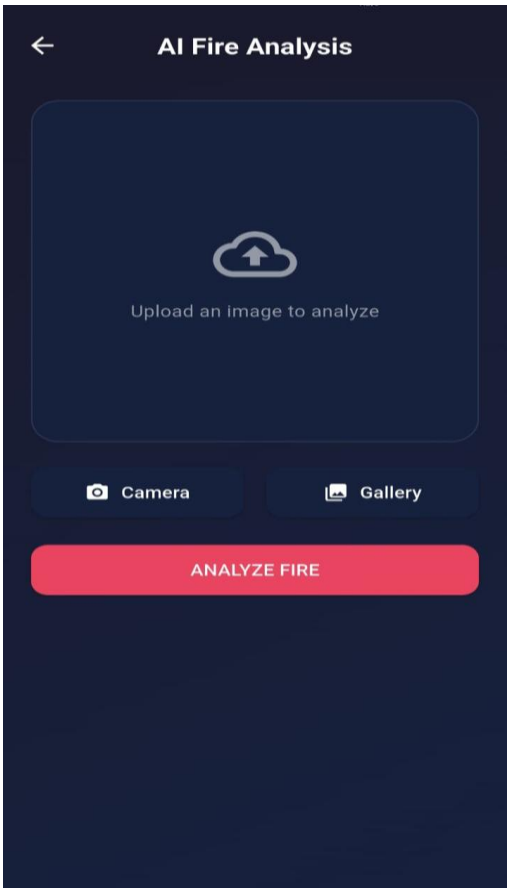

Fig. 5.10 Model Fire Analysis (Upload Screen)

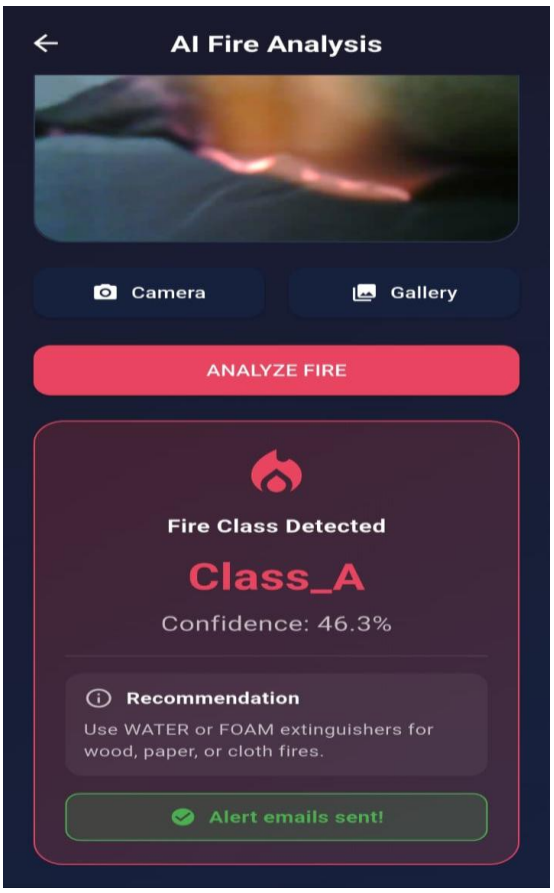

Fig. 5.11 Model Fire Analysis (Result Screen)

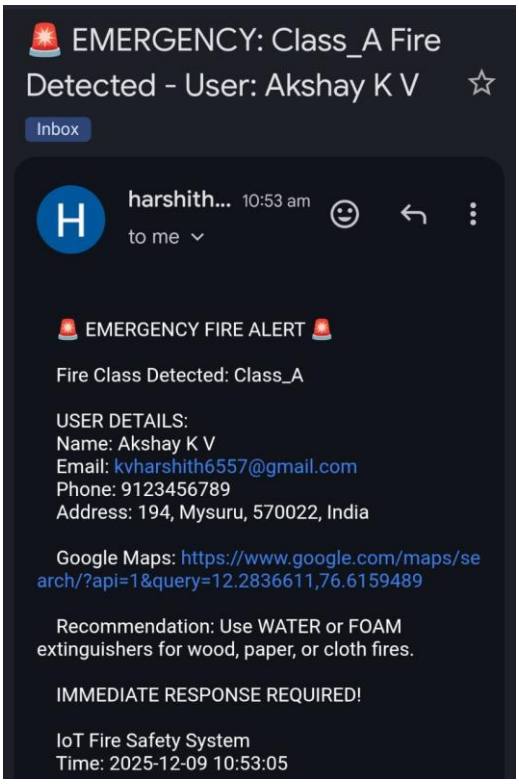

Fig. 5.12 Emergency Alert Email Notification

Chapter 6

CONCLUSION

The proposed IoT-based fire detection system effectively combines multi-sensor data with YOLOv8-based image analysis to provide accurate and timely fire detection. By integrating environmental sensing with visual confirmation, the system reduces false alarms and improves response speed compared to traditional fire alarms. A key advantage is its ability to identify the burning object and automatically classify the fire type, enabling correct extinguisher recommendations for safer and more effective emergency response. The accompanying mobile application enhances situational awareness by delivering real-time alerts, sensor data, location details, and action recommendations, ensuring rapid and coordinated responses across residential, commercial, and industrial environments.

Future work can aim to further improve the robustness and accuracy of the YOLOv8 model, particularly in complex scenarios such as low-light environments and partially obstructed views. The system can be extended to integrate with automated fire suppression systems and smart building infrastructure, allowing actions such as automatic ventilation control, emergency lighting activation, and coordinated evacuation support. Additionally, incorporating cloud-based analytics and predictive fire risk modeling will enable long-term data analysis, early risk assessment, and anticipatory alerts. These enhancements will evolve the system from a reactive detection tool into a proactive, intelligent, and scalable fire safety management solution leveraging AI and IoT technologies.

REFERENCES

[1] S. Debnath, J. Gautam, and S. K. Rai, "IoT Based Smart Home and Office Fire Notification Alert System," in Proc. IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI), 2024.

[2] T. H. Khan, "A Smart Fire Detector IoT System with Extinguisher Class Recommendation Using Deep Learning," IoT, vol. 4, no. 4, pp. 558–581, Nov. 2023.

[3] K. Avazov, A. E. Hyun, A. A. Abrar Sami, A. Khaitov, A. B. Abdusalomov, and Y. I. Cho, "Forest Fire Detection and Notification Method Based on AI and IoT Approaches," Future Internet, 2023.

[4] S. E. C. (Sushmitha E. C.), R. Alageswaran, and R. Ezhilarasie, "IoT Based Fire Detection and Warning System for Surveillance Applications using Deep Learning Model," IEEE, 2023.

[5] C. Jin, T. Wang, N. Alhusaini, S. Zhao, H. Liu, K. Xu, and J. Zhang, "Video Fire Detection Methods Based on Deep Learning: Datasets, Methods, and Future Directions," *Fire*, vol. 6, no. 8, Art. 315, Aug. 2023.

[6] W. Yang, "Deep Learning Method for Real-Time Fire Detection Using YOLOv5s with Squeeze-and-Excitation Module," *Journal of Real-Time Image Processing*, 2024.

[7] D. Gragnaniello, "Fire and Smoke Detection from Videos: A Literature Review," *Expert Systems with Applications*, 2024.

[8] C. Lv et al., "A Lightweight Fire Detection Algorithm for Small Targets Based on an Improved YOLOv5s Network," *Scientific Reports*, 2024.

[9] A. Saleh et al., "Forest Fire Surveillance Systems: A Review of Deep Learning-Based Methods," *Environmental Modelling & Software*, 2023.

[10] Z. Liu, L. Zhang, H. Ma, X. Chen, and M. Zhang, "Research on Lightweight Tunnel Cable Fire Recognition Algorithm Based on Multiscale Features," *Scientific Reports*, vol. 15, Art. 25841, 2025.

[11] Canadian Centre for Occupational Health and Safety (CCOHS), "Fire Extinguishers – Portable," OSH Answers, 2024. Available: https://www.ccohs.ca/oshanswers/safety_haz/fire_extinguishers.html

[12] Khammash, "ClassesOfFire," Kaggle Available: https://www.kaggle.com/datasets/imankhammash/classesoffire (accessed Oct. 18, 2025)

[13] Human Signal, "Label Studio: Data labeling software," Github. [Online]. Available: https://github.com/HumanSignal/label-studio

[14] E. Casas, L. Ramos, E. Bendek, and F. Rivas-Echeverría, "Assessing the Effectiveness of YOLO Architectures for Smoke and Wildfire Detection," {IEEE Access}, vol. 11, pp. 96554–96583, 2023, doi: 10.1109/ACCESS.2023.3312217.}

[15] N. D. Ismail, R. Ramli, and M. N. A. Rahman, "Evaluating YOLOv5s and YOLOv8s for Kitchen Fire Detection: A Comparative Analysis," EMITTER International Journal of Engineering Technology, vol. 12, no. 2, pp. 182–197, 2024. doi: [10.24003/emitter.v12i2.882].